

שיתוף זיכרון

מערכות הפעלה
לינוקס

שיתוף זיכרון

כדי להבין איך יכול להיות מצב שבו שני Processים ניגשים לאותו מקום ב-RAM יש צורך להבין מהו מיפוי זיכרון. לכן חלק ניכר נקדיש למנגנון מיפוי הזיכרון, ולאחר מכן המשיך ההסבר על שיתוף הזיכרון יהיה פשוט.

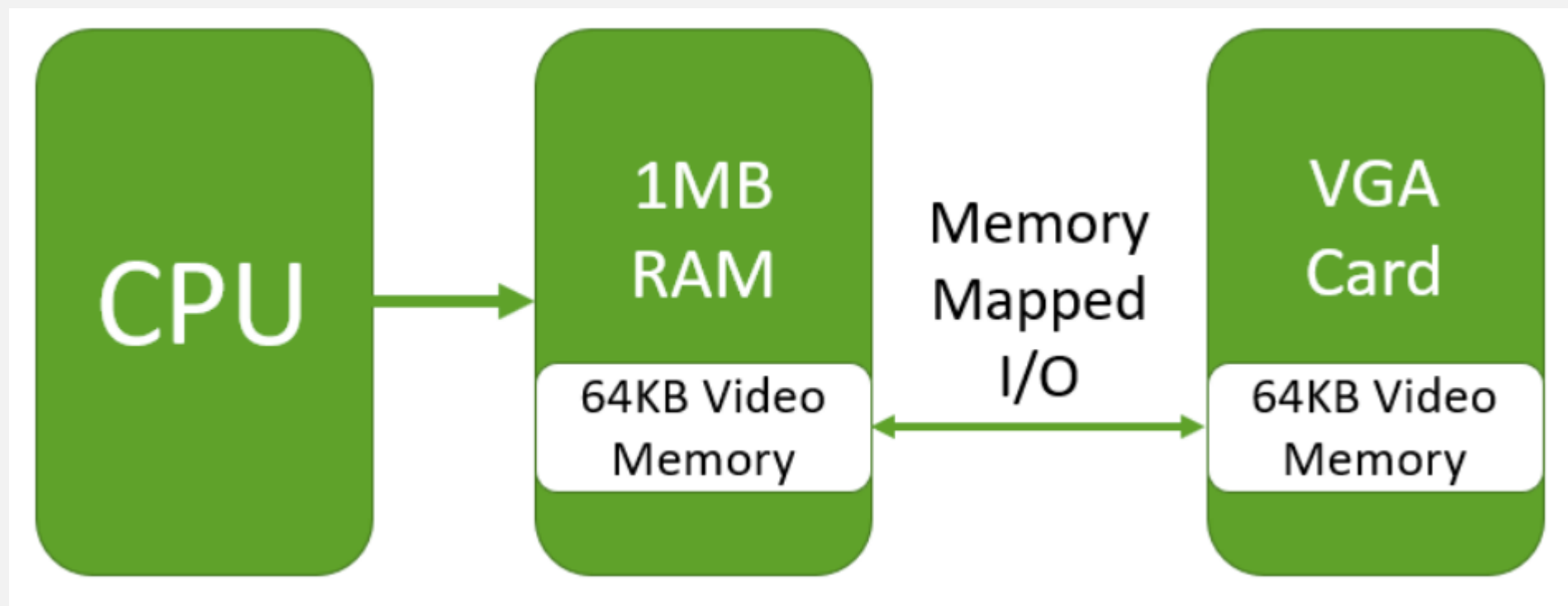
מיפוי זיכרון הוא תהליך שבו נוצר קישור בין זיכרון פיזי ב-RAM לבין זיכרון של התקן חומרה כלשהו. התקן חומרה שבוצע עבורו מיפוי זיכרון ל-RAM נקרא Memory Mapped I/O, כאשר ה-I/O הם קיצור של Input/Output. מה המשמעות של קישור בין שני זיכרונות? הכוונה ליצירת מנגנון שבו כל שינוי בזיכרון אחד יוצר את אותו השינוי בזיכרון אחר. היתרון בשיטה הזו היא שהיא מאפשרת למעבד לפנות בקלות להתקני חומרה, בלי צורך להפעיל Interruptים (פסיקות חומרה) ובלי צורך בפניה ל-Portים ושימוש בפקודות מיוחדות.

מבין התקני החומרה השונים שניתן לבצע אליהם מיפוי זיכרון, התקן חומרה ספציפי הוא התקן לשמירת קבצים כגון הדיסק הקשיח. אם המיפוי מבוצע לקובץ ולא למקרה הכללי יותר של התקן חומרה, הקובץ נקרא פשוט Memory Mapped File.

מיפוי זיכרון קלט/פלט - Memory Mapped I/O

השימוש ב-Memory Mapped I/O קלאסי לכרטיסי גרפיקה, עקב מהירות העדכון הרבה שהם דורשים, לדוגמה במשחקי מחשב, מיפוי זיכרון הוא אפשרות מקובלת. ניקח מערכת שיש בה מעבד, זיכרון RAM וכרטיס גרפי. כל מה שמוצג למסך המשתמש חייב להיות בזיכרון של הכרטיס הגרפי. נמחיש את העקרון באמצעות מעבד ה-8086 ההיסטורי, אך העקרון תקף גם כיום.

למעבד ה-8086 היתה גישה ל-MB1 של זיכרון RAM. לצד המעבד היה מותקן כרטיס גרפי VGA שמסוגל להציג 256 צבעים על מסך שגודלו 320X200 פיקסלים. כל פיקסל דרש בית אחד, לכן גודלו של הזיכרון של כרטיס ה-VGA היה 64KB. כל הזיכרון הגרפי מופה אל אזור מסוים בזיכרון ה-RAM, שהתחיל בכתובת 0xA0000 וגודלו היה 0x10000 (כלומר 64KB).

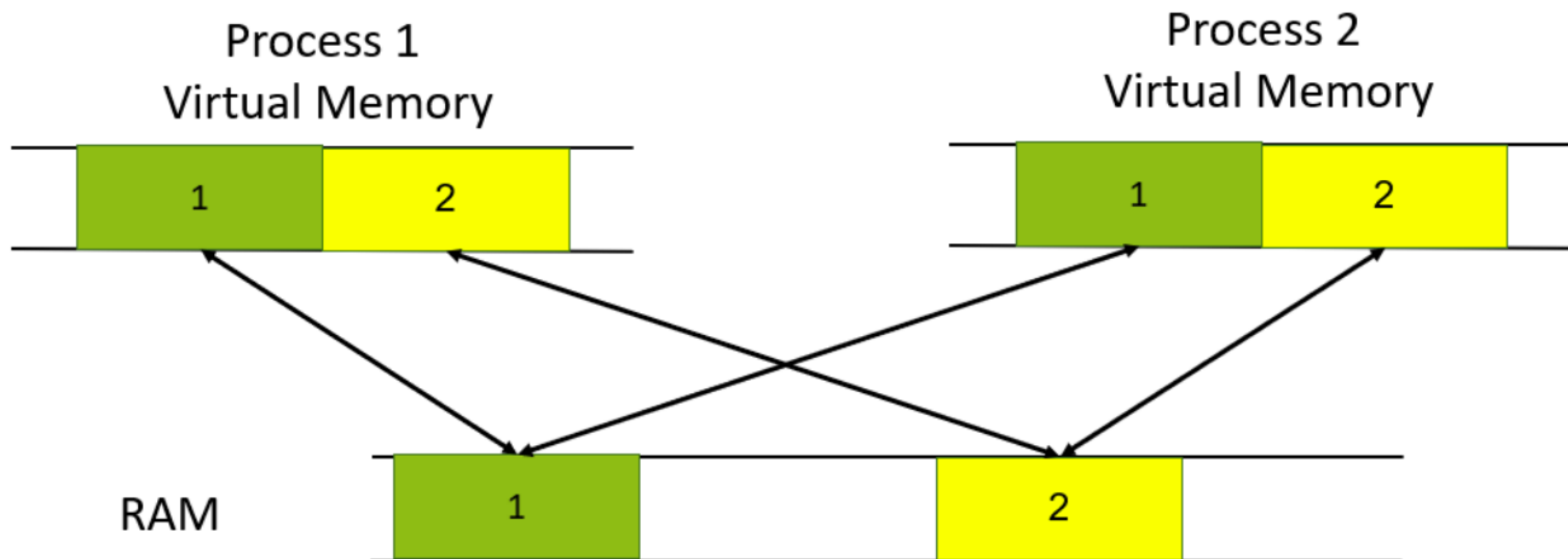


היתרון של מיפוי הזיכרון הוא הפשטות והמהירות: כדי לפנות לזיכרון הוידאו, המעבד פשוט מבצע פקודות העתקה אל אזור הזיכרון שממופה אצלו ל-RAM. אין צורך בפסיקות חומרה איטיות. החיסרון- המיפוי גוזל חלק מזיכרון ה-RAM של המעבד, שלמעשה הפך להיות בלתי שמיש לכל מטרה שאינה תקשורת עם כרטיס הוידאו.

זיכרון משותף - Shared Memory

מיפוי זיכרון יכול להפוך בקלות לשיתוף זיכרון. במצב של שיתוף זיכרון, אותו אזור ב-RAM ממופה לתוך הזיכרון הוירטואלי של שני Processים.

האיור הבא ממחיש את שיתוף הזיכרון, שני קטעי זיכרון ב-RAM ממופים אל שני Process-ים. במצב זה, הזיכרונות של שני ה-Process-ים קשורים ביניהם באותם קטעי זיכרון ב-RAM. אם אחד מה-Process-ים ישנה את הזיכרון בקטעים המשותפים, ה-Process השני יראה את השינוי.



כדי להשיג Shared Memory, יש לפעול לפי השלבים הבאים:

:1 Process

1. פותח קובץ - CreateFile

2. יוצר מיפוי בין הקובץ לזיכרון – CreateFileMapping. כחלק מתהליך המיפוי, נותן שם לאזור בזיכרון. היזכרו בפרמטר האחרון בפונקציה CreateFileMapping, שבדוגמת הקוד נקרא "MyFile"

3. משתמש במיפוי עבור חלק מהקובץ - MapViewOfFile

:2 Process

1. לוקח גישה לאזור הממופה – OpenFileMapping. כדי להשיג גישה, צריך להכיר את שם האזור (בדוגמה - "MyFile")

2. משתמש במיפוי עבור חלק מהקובץ – MapViewOfFile

לקריאה נוספת, מתוך הדרכה של מיקרוסופט:

[https://docs.microsoft.com/en-us/previous-versions/ms810613\(v=msdn.10\)?redirectedfrom=MSDN](https://docs.microsoft.com/en-us/previous-versions/ms810613(v=msdn.10)?redirectedfrom=MSDN)

דוגמאות ל-Shared Memory

נסקור שתי דוגמאות קלאסיות לשיתוף זיכרון: ה-Kernel ו-DLLים.

כפי שלמדנו, חלק ממרחב הזיכרון של כל Process מוקדש לקוד של ה-Kernel של מערכת ההפעלה. דבר זה מאפשר ל-Process לפנות לפונקציות של מערכת ההפעלה.

מה קורה כאשר מבוצע Context Switch אל Process חדש? כיוון שגם ה-Process החדש צריך את אותו הקוד של מערכת ההפעלה במרחב הזיכרון, זה יהיה בזבוז עצום להחליף את כל הדפים ב-RAM שיש בהם את הקוד של מערכת ההפעלה. הרבה יותר פשוט לבצע מיפוי של קוד מערכת ההפעלה שנמצא כבר ב-RAM אל הזיכרון של ה-Process החדש, ובאופן כללי אל כל ה-Processes שרצים. התוצאה היא שקוד של מערכת ההפעלה נמצא Shared Memory בין כל ה-Processes.

מה קורה כאשר תוכניות רבות מייבאות את אותה חבילת קוד? אין הגיון בכך שכל Process יעתיק את חבילת הקוד לתוך ה-RAM.

